Analysis-Guided LLM Code Generation for GPU Kernels: Bridging Polyhedral Analysis and Triton Synthesis

Qin Xiao
University of Leeds
Leeds, United Kingdom
ppnh4382@leeds.ac.uk

Chunwei Xia
University of Leeds
Leeds, United Kingdom

Zheng Wang
University of Leeds
Leeds, United Kingdom

ABSTRACT

Large language models (LLMs) can generate GPU kernel code, but without understanding parallelization constraints they frequently produce incorrect or inefficient implementations. We present TritonPilot, a system that extracts parallelization properties from C loop nests using polyhedral analysis (PET) and LLVM's dependence analysis, then encodes them as structured natural-language guidance in the LLM prompt. On 30 Polybench/C kernels, TritonPilot achieves 100% correctness (vs. 93% without analysis) with 2.15× median speedup over single-threaded C, rising to 28.97× at 8× problem sizes. On 151 TSVC loop kernels, TritonPilot passes 150/151 (99%); at 32 MB per array, the generated GPU kernels achieve 46× median speedup over single-threaded C and 9× over 80-thread OpenMP. The approach generalizes to 8 application kernels from Rodinia and ECP proxy applications, achieving up to 25.8× speedup.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel computing methodologies*.

KEYWORDS

LLM code generation, GPU kernels, Triton, polyhedral analysis

ACM Reference Format:
Qin Xiao, Chunwei Xia, and Zheng Wang. 2026. Analysis-Guided LLM Code Generation for GPU Kernels: Bridging Polyhedral Analysis and Triton Synthesis. In *Proceedings of Workshop on AI for High Performance Compiler Construction (AI4HPCC '26).* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

GPU programming remains a significant barrier to adopting hardware accelerators for scientific computing. Writing efficient GPU kernels requires expertise in thread-level parallelism, memory coalescing, and synchronization—skills that domain scientists often lack. OpenAI's Triton [1] simplifies GPU programming with a block-level abstraction, but generating correct Triton code still requires understanding which loop dimensions are safe to parallelize, how data dependences constrain execution order, and when synchronization barriers are needed.

Recent work has shown that LLMs can generate functional GPU code [2, 3], but they lack the ability to perform precise dependence analysis. As a result, LLM-generated GPU code frequently suffers from:

(1) **Correctness failures**: data races from parallelizing dimensions with carried dependences;
(2) **Performance anti-patterns**: serial launches, scalar loops, or excessive kernel invocations;
(3) **Synchronization errors**: missing memory barriers between phases in stencil computations.

We present TritonPilot, which integrates classical compiler analysis into the LLM prompting pipeline. TritonPilot uses PET [4] and LLVM's dependence analysis [5] to extract parallelization properties, then translates these into structured guidance that steers the LLM toward correct Triton implementations. The key insight is that *analysis results can be communicated to LLMs as domain-expert advice*, enabling the model to make informed parallelization decisions without performing dependence analysis itself. Our contributions:

- A pipeline that extracts parallelization constraints via polyhedral and LLVM analysis and encodes them as structured LLM prompt guidance (§2);
- Evaluation on 30 Polybench/C kernels (100% pass, 2.15× median), 151 TSVC kernels (99% pass, 9× median over 80-thread OpenMP at 32 MB), and 8 application kernels from Rodinia/ECP proxy applications (§3).

2 METHOD

Given a C loop nest, TritonPilot proceeds in four stages: (1) static analysis, (2) prompt construction, (3) LLM generation and validation, and (4) iterative refinement.

Static Analysis. PET extracts a polyhedral representation from which we compute parallelizable dimensions with validity constraints, WAR dependences with loop-level scoping, scalar expansion opportunities, and reduction types. When PET cannot handle a construct (e.g., non-affine subscripts), we fall back to LLVM's `DependenceAnalysis` pass, parsing direction vectors to determine which loop levels carry dependences.

Prompt Construction. Analysis results are translated into structured natural-language sections appended to the LLM prompt. Five principal categories of guidance are generated:

AI4HPCC '26, July 6–9, 2026, Belfast, Northern Ireland, UK
2026. ACM ISBN 978-x-xxxx-xxxx-x/YY/MM. . . $15.00
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Parallelization dimensions: Each loop dimension is classified as valid (safe to parallelize), invalid with a specific reason, or reduction-safe. The prompt presents valid options with grid configurations and flags unsafe dimensions with explanations:

```
Option 1: Parallelize dim 'j' (innermost)
  - Grid: triton.cdiv(NY, BLOCK)
  - Each CTA processes BLOCK elements of j
  - Issue: None (fully parallel)
Option 2: Parallelize dim 'i' (outermost)
  - Issue: Dependencies carried on 'i'
```

WAR dependences: PET computes exact WAR sets via polyhedral dependence analysis; when PET fails, we parse LLVM direction vectors and map them to loop levels. Three cases are distinguished—no WAR (in-place update), WAR with parallel resolution (`B = A.clone()` and read from clone while writing to original), and WAR requiring sequential execution (`grid=(1,)` with explicit loops inside the kernel).

Multi-phase decomposition: Many kernels contain multiple computational phases with inter-phase dependences (e.g., `correlation`: mean → normalization → correlation matrix). The analysis detects phases via cross-phase write conflicts and recommends splitting into separate kernel launches connected by a Python driver loop. This is critical: LLMs frequently attempt to fuse all phases into a single kernel, causing incorrect results or serial execution.

Reduction patterns: Reduction operations are mapped to Triton primitives (`tl.sum()`, `tl.dot()`). For kernels with opposing reductions (e.g., `bicg`: row reduction on A and column reduction on A^T), the guidance recommends fusing both reductions with row-major iteration for coalesced memory access.

Stencil synchronization: For stencil computations, the analysis detects cross-phase data dependences within a single CTA. Triton's vectorized stores go to L1 cache, and subsequent loads from overlapping addresses may read stale data. The guidance inserts `tl.debug_barrier()` calls between phases and includes a pre-validation check that rejects generated code missing these barriers before executing the expensive GPU test.

Generation and Refinement. The prompt is sent to Claude Sonnet 4 [7], which generates a Triton kernel implementation. The generated code is tested against a C reference compiled as a shared library (`-O0` to preserve reference semantics): both implementations execute on identical random inputs, and outputs are compared element-wise with per-kernel tolerances that account for FP32 accumulation order differences.

If validation fails, the system classifies the error and constructs a targeted retry prompt. For numerical mismatches, the feedback includes the maximum absolute error and suggests index-bounds checking. For performance issues (<0.1× speedup), the feedback identifies anti-patterns (serial grid, scalar loops, excessive launches). Up to 10 attempts are made, with a context reset at attempt 6 to escape fixation on broken approaches.

TritonPilot is implemented in approximately 14,000 lines of Python. The analysis frontend comprises 18 modules built on PET (which uses isl [8] for polyhedral operations) and LLVM 17 for fallback dependence analysis.

Table 1: Polybench/C results. WA = with analysis, NA = no analysis.

Metric	WA	NA
Pass rate	30/30 (100%)	28/30 (93%)
Median speedup	2.15×	1.06×
Mean speedup	3.44×	1.52×
Kernels > 1×	20/30	14/28
Head-to-head (28 common):		
WA wins (>10% faster)	12	
NA wins (>10% faster)	6 (3 race conditions)	
Ties (within 10%)	10	

Table 2: Polybench scaling: 1× to 8× problem sizes (N: 60→480, matrices: 120→960). "ST" = single-threaded C; "OMP" = 80-thread OpenMP.

Metric	1×	8× (ST)	8× (OMP)
Median speedup	2.15×	28.97×	7.98×
Kernels > 1×	20/30	22/26	21/27
Max speedup	12.36×	1826×	1464×

3 EVALUATION

All experiments run on an NVIDIA RTX 3090 GPU (24 GB, 936 GB/s) with a dual-socket Xeon Gold 5218R (80 cores, ∼140 GB/s), PyTorch 2.5.1, Triton 3.1.0, CUDA 12.4. The LLM is Claude Sonnet 4 [7]. Polybench/application speedups are vs. single-threaded C (`-O2`); TSVC additionally reports speedup vs. an 80-thread OpenMP baseline (`-fopenmp`). All speedups are kernel-only with data pre-staged on GPU, averaged over 10–50 iterations after warmup.

3.1 Polybench/C (30 Kernels)

Table 1 summarizes results at standard (1×) problem sizes. All 30 kernels pass with analysis guidance; without it, two stencil kernels fail due to parallelizing dimensions with carried dependences. The 2.15× median speedup is limited by Polybench's small defaults ($N = 60$–120), where GPU launch overhead dominates. Compute-bound kernels show strong speedups (`heat_3d`: 12.36×, `deriche`: 7.44×, `covariance`: 7.40×).

Of 6 apparent NA wins, 3 (`lu`, `ludcmp`, `bicg`) contain confirmed race conditions—the NA code uses `grid=(N,)` for LU decomposition where row i reads from row k ($k < i$) that may not yet be computed by its CTA, producing up to 10^{33} variation across runs. The remaining 3 NA wins reflect nondeterministic LLM code quality, not structural differences.

Scaling. At 8× sizes (Table 2), median speedup rises to 28.97× over single-threaded C. Against 80-thread OpenMP, the GPU still achieves 7.98× median, confirming genuine

Table 3: Impact of analysis-triggered guidance categories. "Before" and "After" are speedups at 1× sizes except where noted.

Guidance	Kernel	Before	After
Multi-phase split	correlation	0.19×	2.50×
WAR sequential	nussinov	FAIL	PASS
Triangular grid	lud†	0.14×	13.18×
Fused reductions	bicg	1.04×	3.51×
Stencil barriers	jacobi_1d	0.13×	1.78×
IIR phase fusion	deriche	0.28×	7.19×
Tridiag. sweep	adi	0.24×	2.27×

†Rodinia benchmark.

advantage beyond CPU parallelism. Matrix-multiply kernels reach 1464× (`3mm`) even vs. OpenMP. Four kernels time out (inherently sequential, `grid=(1,)`).

Guidance Impact. Table 3 shows the impact of individual guidance categories developed through iterative analysis of failure modes. Each category is triggered by analysis properties (e.g., cross-phase dependences, triangular access patterns), not per-kernel heuristics. The largest improvements come from multi-phase decomposition (correlation: 13× improvement) and the triangular grid recommendation (lud: 94× improvement). The IIR phase-fusion guidance detects that the forward, backward, and combination phases of a deriche filter operate on independent rows/columns, fusing all three into a single kernel with `grid=(W,)` or `grid=(H,)` (2 launches instead of 6). The tridiagonal sweep guidance identifies that ADI has one parallelizable spatial dimension and one with sequential recurrence, recommending `grid=(N-2,)` with one CTA per row. At 8× sizes, a 2D stencil threshold adjustment (multi-CTA grids when ≥2 valid dimensions exist) improved `jacobi_2d` to 36.82× and `fdtd_2d` to 15.43×.

Limitations. Eleven kernels achieve <1× at standard sizes. Five are inherently sequential (`durbin`: 0.43×, `gramschmidt`: 0.35×, `trisolv`: 0.58×, `ludcmp`: 0.36×, `nussinov`: 0.06×), where loop-carried dependences on most dimensions limit parallelism; the analysis correctly identifies these constraints and the LLM generates working but slow GPU implementations.

Two kernels (`jacobi_2d`: 0.71×, `fdtd_2d`: 0.34×) have parallelizable spatial dimensions but at Polybench's default sizes (N=20–30), launching 2–4 kernels per timestep×20–40 timesteps creates 80–160 launches that dominate execution time. At 8× sizes, these same kernels achieve 36.82× and 15.43×, confirming that the parallelization strategy is correct and the 1× regression is purely a problem-size effect. These <1× results are *correct* failures—preferable to the incorrect parallelization that the NA baseline produces for `jacobi_2d` and `seidel_2d`.

Table 4: TSVC results at three data sizes. "ST" = single-threaded C (`-O2`), "OMP" = OpenMP C (`-O2 -fopenmp`, 80 threads). Pass rate: 150/151 (99.3%).

	32 KB	32 MB	3200 MB
Per-array size	128 KB	32 MB	3.2 GB
Kernels tested	84	27	15
GPU/ST median	0.11×	46.3×	73.2×
GPU/ST >1×	1/84	25/27	14/15
GPU/OMP median	0.13×	9.0×	14.2×
GPU/OMP >1×	1/84	24/27	14/15

Table 5: Application kernel results. "—" = NA failed all attempts.

Source	Kernel	WA	NA
Rodinia	SRAD	24.86×	3.50×
Rodinia	lud	13.18×	15.52×
Rodinia	gaussian	3.35×	3.61×
Rodinia	lavaMD	1.47×	0.56×
Rodinia	hotspot	1.05×	1.22×
Rodinia	pathfinder	0.11×	—
miniMD	LJ force	25.82×	23.00×
miniFE	SpMV	8.95×	7.27×

3.2 TSVC (151 Kernels)

To evaluate breadth, we apply TritonPilot to the 151-kernel TSVC (Test Suite for Vectorizing Compilers) [15] benchmark, which covers linear traversals, reductions, indirect addressing, control flow, and loop-carried dependences. Unlike Polybench's multi-dimensional stencils, TSVC kernels are predominantly 1D loops operating on flat arrays, testing whether the analysis pipeline handles simpler but diverse loop patterns.

Table 4 shows TSVC results across three data sizes. TritonPilot passes 150 of 151 kernels (99.3%), with 120 on the first attempt. At the default 32 KB per array (N=32,000), GPU launch overhead dominates and the median speedup is 0.11×—neither the GPU nor multi-threaded CPU can amortize overhead at this scale.

At 32 MB (N=8M), the GPU achieves 46.3× median over single-threaded C and 9.0× over 80-thread OpenMP. At 3200 MB (N=800M), the GPU advantage grows to 73.2× (ST) and 14.2× (OMP), as the GPU's memory bandwidth (936 GB/s on RTX 3090) decisively outperforms the dual-socket Xeon's bandwidth (∼140 GB/s). Serial kernels with loop-carried dependences are excluded from the 32 MB and 3200 MB benchmarks since neither CPU nor GPU can parallelize them.

3.3 Application Kernels

To evaluate generalization beyond synthetic benchmarks, we apply TritonPilot to 6 Rodinia kernels [13] and 2 ECP proxy application kernels [14]—CSR sparse matrix-vector multiply

(SpMV) from miniFE and Lennard-Jones force computation from miniMD (Table 5). All 8 pass correctness with analysis guidance (6 on the first attempt).

SRAD (24.9×) has three computational phases per iteration; the analysis detects cross-phase dependences and recommends splitting into separate kernels with parallel grids, yielding 7.1× improvement over the NA baseline (3.5×). The LJ force kernel from miniMD computes pairwise forces using a full neighbor list; the outer loop over atoms is embarrassingly parallel. With analysis guidance the LLM iterates only actual neighbors (25.8×), while the NA version wastes cycles on padded entries (23.0×). lavaMD improves from 0.56× (NA) to 1.47× (WA) as analysis identifies the parallelizable particle dimension, while the NA baseline fails with type-mismatch errors on 4 of 5 attempts. SpMV achieves 8.95× with analysis vs. 7.27× without, as guidance helps select better block sizes for the row-parallel pattern. For lud and hotspot, both versions produce near-identical Triton times; the speedup differences reflect C reference timing variance.

4 RELATED WORK

LLM Code Generation for GPUs. Chen et al. [2] evaluate ChatGPT on CUDA kernel generation, finding frequent correctness issues including data races and incorrect memory management. KernelBench [3] benchmarks LLM-generated Triton/CUDA on 250 tasks, reporting 20–40% functional correctness without guidance. Our work differs fundamentally: rather than relying on the LLM's implicit knowledge of parallelism, we provide explicit compiler-derived constraints, achieving 100% correctness on Polybench/C and 99% on TSVC.

Polyhedral Compilation and Auto-Scheduling. Polyhedral frameworks such as Pluto [11] and PPCG [12] automatically generate parallel code from affine loop nests. PPCG targets CUDA directly but is limited to affine programs and produces low-level code that lacks Triton's block-level optimizations (automatic memory coalescing, shared memory tiling). TVM [9] and Halide [10] use auto-scheduling with learned cost models for tensor operations. TritonPilot is complementary: rather than replacing the compiler, it uses polyhedral analysis to extract *what* parallelism is safe, then delegates the *how* to the LLM, which can handle non-affine constructs, irregular access patterns, and high-level Triton abstractions that polyhedral compilers cannot target directly.

5 CONCLUSION

We presented TritonPilot, a system that bridges polyhedral compiler analysis with LLM-based GPU kernel generation. By translating parallelization constraints, dependence information, and memory access patterns into structured prompt guidance, TritonPilot enables an LLM to generate correct and efficient Triton kernels for all 30 Polybench/C benchmarks (2.15× median), 150 of 151 TSVC kernels (9× median over 80-thread OpenMP at realistic data sizes), and 8 application kernels with up to 25.8× speedup.

The ablation study reveals two key findings: (1) analysis guidance prevents correctness failures that stem from unsafe parallelization of dependent loop dimensions, and (2) unguided code frequently contains latent race conditions—producing results that pass tolerance checks by chance but exhibit nondeterministic errors at larger scales. Three of six apparent NA "wins" in Polybench were confirmed to contain such race conditions via repeated execution.

For application kernels, analysis guidance is essential for complex multi-phase computations (SRAD: 7× improvement over unguided) and beneficial even for naturally parallel patterns (LJ force: guidance promotes efficient neighbor-list traversal). No application kernel shows a Triton-time regression with analysis guidance enabled.

Future work includes extending the analysis to non-affine programs, supporting multi-GPU parallelization, and integrating autotuning for block size selection.

REFERENCES

[1] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proc. MAPL*, 2019.

[2] Y. Chen, S. Huang, D. Zheng, et al. ChatGPT for CUDA programming: Evaluating LLM-generated GPU code. *arXiv preprint arXiv:2404.12000*, 2024.

[3] A. Ouyang, S. Zheng, et al. KernelBench: Can LLMs write efficient GPU kernels? *arXiv preprint arXiv:2404.12489*, 2024.

[4] S. Verdoolaege and T. Grosser. Polyhedral extraction tool. In *Proc. IMPACT*, 2012.

[5] C. Lattner and V. Adve. LLVM: A compilation framework for lifelong program analysis & transformation. In *Proc. CGO*, 2004.

[6] L.-N. Pouchet. Polybench/C: The polyhedral benchmark suite, version 4.2.1. https://web.cs.ucla.edu/~pouchet/software/polybench/, 2016.

[7] Anthropic. Claude Sonnet 4 model card. https://www.anthropic.com/claude, 2025.

[8] S. Verdoolaege. isl: An integer set library for the polyhedral model. In *Proc. ICMS*, 2010.

[9] T. Chen, T. Moreau, Z. Jiang, et al. TVM: An automated end-to-end optimizing compiler for deep learning. In *Proc. OSDI*, 2018.

[10] J. Ragan-Kelley, C. Barnes, A. Adams, et al. Halide: A language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines. In *Proc. PLDI*, 2013.

[11] U. Bondhugula, A. Hartono, J. Ramanujam, and P. Sadayappan. A practical automatic polyhedral parallelizer and locality optimizer. In *Proc. PLDI*, 2008.

[12] S. Verdoolaege, J. Carlos Juega, A. Cohen, et al. Polyhedral parallel code generation for CUDA. *ACM TACO*, 9(4), 2013.

[13] S. Che, M. Boyer, J. Meng, et al. Rodinia: A benchmark suite for heterogeneous computing. In *Proc. IISWC*, 2009.

[14] M. A. Heroux, D. W. Doerfler, P. S. Crozier, et al. Improving performance via mini-applications. Sandia National Laboratories Technical Report SAND2009-5574, 2009.

[15] D. Callahan, J. Dongarra, and D. Levine. Vectorizing compilers: A test suite and results. In *Proc. Supercomputing*, 1988.